



Kai: Typed Decision Models as a Control Plane for Agentic Systems

Decisions as versioned programs
under deterministic policy,

with sensitivity and refresh for auditable studies

Hanzo AI Research

Hanzo AI

2026-09-24 Weights at [hanzoai/kai-1](#) b50502c2

Kai: Typed Decision Models as a Control Plane for Agentic Systems

Decisions as versioned programs under deterministic policy,
with sensitivity and refresh for auditable studies

Hanzo AI Research*
Hanzo Industries Inc (Techstars '17)
Los Angeles, California
<https://hanzo.ai>

2026-09-24 Weights at `hanzoai/kai-1` b50502c2

Abstract

An agent makes many small decisions for every task it performs: whether a command may run, which model a subtask needs, which context the next turn should see, whether the work is finished. Most agents put each of these to a generative model and parse its prose. We describe Hanzo Decision, which moves them to a discriminative plane. A decision is a typed question (**choice**, **score** or **noul**) over explicit state, and its answer is a probability distribution over criteria the caller enumerated, computed in one encoder forward pass with no generated text. Kai is the model family that answers them: three checkpoints on ModernBERT and mmBERT encoders, published as `hanzoai/kai-1`, `kai-1-multilingual` and `kai-1-agent` and derived from the open Laya checkpoints. Decisions are grouped into versioned Decision Programs whose thresholds are bound to a calibration and whose output runs in shadow, advisory or enforced mode. A deterministic policy remains the authority. Verdicts form the lattice `ALLOW < ASK < DENY` and the effective verdict is the join of the policy's and the model's, so no model output, including one induced by adversarial state, can loosen a policy verdict. The risk of enforcing a model is then friction alone, and friction can be counted from shadow records before a program is promoted. The public interface, `POST /v1/decisions`, is wire-compatible with OpenRouter's Decisions API and routes by model name to native Kai or to TypeSafe's Jev. We carry the same objects to studies: a `DecisionPackage` records a trade study as typed answers over evidence, reports which weights and which answers would flip its recommendation, and refreshes only the answers whose evidence changed. In a recorded run of the three bootstrap checkpoints on a command that sends a private SSH key to a remote host, two ranked `ALLOW` first and none placed more than 0.25 on `DENY`. That result is why the model is advisory and the policy is not.

*Corresponding author: research@hanzo.ai

Contents

1	Introduction	4
2	Typed decisions	6
2.1	What an answer is sure of	6
3	Kai	7
3.1	Input	7
3.2	Encoder and head	7
3.3	The family	8
3.4	Routing between checkpoints	8
3.5	Provenance	9
3.6	The Rust runtime	9
4	Policy is the authority	9
4.1	The verdict lattice	9
4.2	Abstention and escalation	10
4.3	Modes, and why promotion is measurable	10
4.4	Constraints and preferences	11
5	Decision Programs and the wire	11
5.1	Decision Programs	11
5.2	The decision record	12
5.3	Testing a program	12
5.4	POST /v1/decisions	13
5.5	Where it runs	13
5.6	First deployment: the coding agent in shadow	13
6	Decisions as programs for studies	14
6.1	The study as a program	14
6.2	The DecisionPackage	15
6.3	What flips the decision	15
6.4	Refresh	17
6.5	Evidence and time	17
7	Evaluation	17
7.1	The recorded run	18
7.2	What will be measured, and how	19
8	Related work	20
9	Limitations	21
10	Conclusion	21
A	The wire	23
B	A Decision Program	24

1 Introduction

A coding agent is a loop around a generative model with a handful of tools. The design notes on Jev [1] argue that the leverage in such a system lies less in the loop than in what the loop feeds the model each time it turns, and that underneath every feature of a capable harness sits a small decision asked at high frequency. Should this chunk of context be visible to the next turn, and in how much detail? Can this subtask leave the frontier model? Which of several hundred tools fits the stated intent? May this command run? A session asks such questions thousands of times.

Most agents answer them by asking a generative model and reading its prose, or by a fixed rule. The generative answer costs a generation, has to be parsed, and carries no probability that a caller could compare with a threshold. The fixed rule is cheap and auditable but blind to everything its author did not foresee. A *typed decision* sits between the two. The caller states the question, enumerates the admissible answers with a description of each, and passes the state the answer should depend on. The model returns a probability distribution over exactly those answers. The output can be validated against its type, compared with a threshold, logged with the state it was computed from, and replayed.

Three planes. Hanzo separates an agentic system into three planes with different kinds of authority.

- The *generative plane* writes: code, plans, explanations, drafts. In Hanzo this is the Zen model family and the models it routes to.
- The *decision plane* classifies: it answers typed questions about explicit state with calibrated distributions. This is Kai.
- The *policy plane* decides what is permitted. It is deterministic, written by people, and has the last word.

The decision plane is advisory to the policy plane by construction (Section 4): a model’s verdict can make an outcome stricter than the policy’s and never looser. The generative plane is advisory to both where a decision governs it, such as which context it sees or whether its command runs.

The decision as a program. Most AI systems treat a decision as text: evidence goes into a prompt, a recommendation comes out as prose, and what produced it is not recorded in any form a machine can check (Figure 1, left). Hanzo treats the decision as a program. Generative models help build the program and reason inside it. Typed decision models make the bounded judgments within it. Deterministic code does the arithmetic. Policy and people hold authority. Every run produces a reproducible record, a Decision Package, that can be replayed, compared with another run, and refreshed when its evidence changes (Figure 1, right). A typed decision is the primitive this requires, between ordinary deterministic code and free-form generation. It is more than a cheap classifier: it is the step that gives a judgment a type, a probability and a record.

The layers are separable, and each builds on the one below it. Jev put typed questions inside software. Kai answers them natively, as open weights on Hanzo’s own runtime. Hanzo Decision adds versioned programs, policy, evidence, provenance, refresh and replay. Hanzo’s cloud deploys it as infrastructure.

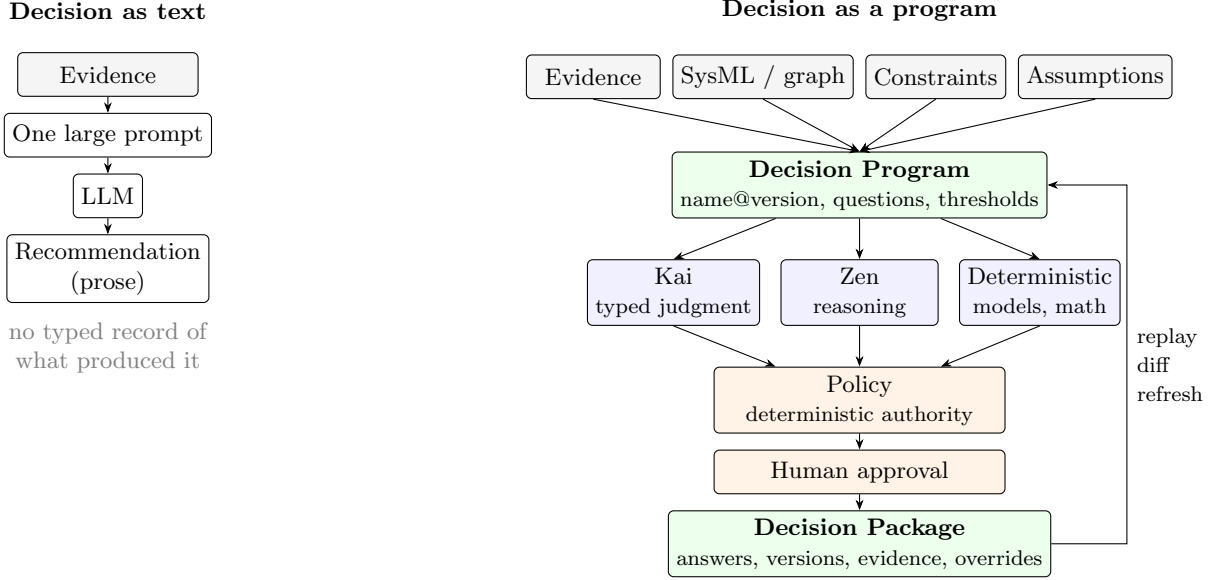


Figure 1: Left: the usual shape, in which one generative call turns evidence into a recommendation and nothing typed records how. Right: Hanzo Decision. A versioned program states the questions; Kai answers the bounded judgments, Zen the open reasoning, and deterministic models the arithmetic; policy and a person hold authority; every run produces a package that can be replayed, compared with another run, and refreshed.

The system. Hanzo Decision is the capability; `POST /v1/decisions` is its only public interface; Kai is the model family behind it. The interface’s wire format is OpenRouter’s Decisions API [18], so a client written for TypeSafe’s Jev [22] works unchanged, and the `model` field chooses the backend: a `kai-*` name is answered natively and a `typesafe/jev-*` name is forwarded to OpenRouter. The Kai checkpoints are the three open Laya checkpoints [4], republished under `hanzoai/` at pinned revisions with byte-identical weights, and a native runtime for them is being written in Rust on Hanzo’s tensor library so that one binary can serve them on a CPU without Python.

Contributions. Each item is marked with its status: [\[measured\]](#) for a figure from a run we made, [\[reported\]](#) for a figure another party published, [\[in the code\]](#) for code or weights that exist at the revision named, and [\[proposed\]](#) for a design that is not yet on a main branch.

1. A definition of typed questions and of the uncertainty an answer reports, and the observation that two existing systems report different quantities under the name `confidence`, so a threshold tuned on one does not transfer to the other (Section 2).
2. The Kai architecture as ported to Rust: the input layout, the typed head, temperature by question type and option count, and the act head, together with the provenance of each checkpoint (Section 3). The weights are [\[in the code\]](#); the runtime is [\[proposed\]](#).
3. Policy precedence: a verdict lattice, abstention to ASK, three execution modes, and a proof that enforcing a model under the join can only add friction, which makes promotion a question answerable from shadow records (Section 4). [\[proposed\]](#)
4. Decision Programs, the seven programs an agent loop needs, the decision record, and the model-agnostic wire (Section 5). [\[proposed\]](#)

5. Decision-as-a-program for studies: the `DecisionPackage`, closed forms for the smallest weight change and the smallest answer change that flip a recommendation, and refresh that recomputes only answers whose evidence changed (Section 6). [\[proposed\]](#)
6. Measurements of the three bootstrap checkpoints on nine recorded cases, and the protocol by which the rest of the system will be measured (Section 7). [\[measured\]](#)

What this paper does not claim. It reports no accuracy of Kai on Hanzo’s traffic, because no Kai checkpoint has yet been trained on it. It reports no latency for the Rust runtime, because that runtime has not yet been measured against the reference. It claims no cost saving from moving decisions off the generative plane; Section 7.2 says how that will be measured. Throughput and accuracy figures for the upstream checkpoints are quoted from their authors and marked as such.

2 Typed decisions

Definition 1 (Typed question). A typed question is a triple $q = (\tau, \iota, C)$ of a type $\tau \in \{\text{choice}, \text{score}, \text{noul}\}$, an instruction ι in natural language, and an ordered list of criteria $C = (c_1, \dots, c_k)$, each a label with a description.

- A **choice** question selects one label from $k \geq 1$ unordered options.
- A **score** question selects one of k ordered levels $0, \dots, k-1$, each with a description.
- A **noul** question is a predicate: $k = 2$, with criteria (**false**, **true**) whose descriptions the caller may override.

Definition 2 (Answer). An answer to q over a state s is a distribution $p \in \Delta^{k-1}$ over the criteria. It is reported by type: a **choice** answer reports $\arg \max_i p_i$; a **score** answer reports the expected level $\bar{\ell} = \sum_i i p_i$ together with a legend mapping levels to descriptions; a **noul** answer reports p_{true} . Every answer also carries the full distribution.

The state is whatever the answer should depend on: a message, a JSON object describing a shell command and its working directory, a transcript, a document. A question never sees anything else, which is what makes it replayable: the pair (question, state) and the model revision determine the distribution.

2.1 What an answer is sure of

Three summaries of p are in use, and they are different numbers.

- The probability of the reported answer, $p_{\max} = \max_i p_i$. We call it **answer.confidence**. When the model is calibrated it is the probability that the reported answer is right, which is the quantity a risk threshold should be written against.
- The rescaled maximum

$$\kappa = \frac{k p_{\max} - 1}{k - 1}, \quad k \geq 2, \tag{1}$$

which is 0 at the uniform distribution and 1 at a point mass. Jev and OpenRouter report this as **confidence** [\[4, 18\]](#).

- One minus normalised entropy, $h = 1 - H(p)/\ln k$, which the Laya runtime reports as **confidence** on **choice** and **score** answers [4].

Proposition 1. *For fixed k , a threshold on κ is a threshold on $p_{\max}$: $\kappa \geq t \iff p_{\max} \geq (1 + t(k - 1))/k$. The implied threshold on $p_{\max}$ depends on k .*

Proof. Equation (1) is affine and increasing in $p_{\max}$ for $k \geq 2$; solve $\kappa = t$ for $p_{\max}$. $\square$

So $\kappa \geq 0.5$ asks for $p_{\max} \geq 0.75$ on a **noul** question, 0.67 on a three-way choice and 0.55 on a ten-way choice. A single threshold applied across questions of different arity is a different risk level on each. The entropy summary h depends on the whole distribution, not only its maximum, and orders answers differently again. In our recorded run (Section 7) the English checkpoint answered an urgency question with $p = (0.039, 0.150, 0.811)$ over three levels: $p_{\max} = 0.811$, $\kappa = 0.717$ and $h = 0.471$ [measured]. A gate at 0.5 passes this answer under one definition and abstains under the other.

The convention we fix. Every backend behind `/v1/decisions` reports **confidence** as κ , so a threshold written for Jev means the same thing when Kai answers, and adds **answer_confidence** = $p_{\max}$ as a separate field. The entropy summary is kept in the runtime for diagnostics and is not on the wire. A Decision Program writes its abstention thresholds against κ and its **noul** thresholds against p_{true} , and records the calibration those thresholds were set against (Section 5). The Rust runtime implements this convention and its tests assert Equation (1) on every backend [proposed].

3 Kai

Kai is a cross-encoder with a typed head. It reads the question, the options and the state together and scores each option at a marker token, so every option is judged in the context of the others and of the state, in one forward pass, without generating text. The description below is of the architecture as the Rust port implements it, which reproduces the reference PyTorch implementation of the upstream checkpoints [4].

3.1 Input

Each row of a batch is one question over one state:

[CLS] $\langle \tau \rangle$ question: ι [SEP] [MASK] c_1 [MASK] $c_2 \dots$ [MASK] c_k [SEP] s [SEP]

Each option is written as its label and description after a marker token; a **score** level is written as **level i** : followed by its description. The positions $m_1, \dots, m_k$ of the markers are recorded with a mask over the option slots. All questions of a request are collated into one batch, so a request with several questions over one state is still a single forward pass. The options share a token budget, **head_max_len**, of 192 tokens on the English checkpoint and 256 on the other two; beyond it options are trimmed, and a question whose options cannot fit is rejected rather than truncated silently [4].

3.2 Encoder and head

Let E be the encoder and $H = E(x) \in \mathbb{R}^{L \times d}$ its output. The head adds a learned type embedding $e_\tau \in \mathbb{R}^d$ to every position and applies two pre-norm transformer encoder layers T_1, T_2 (ReLU feed-forward of width $4d$, $d/64$ heads, LayerNorm $\epsilon = 10^{-5}$, a key padding mask from the attention mask):

$$\tilde{H} = T_2(T_1(H + \mathbf{1}e_\tau^\top)). \quad (2)$$

A scorer reads the head’s state at each marker:

$$z_i = w_2^\top \text{GELU}(W_1 \text{LN}(\tilde{H}_{m_i}) + b_1) + b_2, \quad i = 1, \dots, k, \quad (3)$$

and option slots the row does not have receive $z_i = -10^4$, so they take no mass. The distribution is a tempered softmax,

$$p_i = \frac{\exp(z_i/T_{\tau,k})}{\sum_j \exp(z_j/T_{\tau,k})}, \quad (4)$$

where $T_{\tau,k}$ is a temperature fitted after training for the question type and, where the checkpoint ships one, for the bucket of option counts k falls in. The runtime confines every temperature to $[0.5, 5]$: the English checkpoint ships a fitted value of 0.10 for choices of eleven or more options, which would sharpen those distributions tenfold, and the multilingual checkpoint ships $T = 1$ for every type, which is to say no fit at all [measured].

The act head. A second head reads the [CLS] state together with four features of the option distribution, $\phi(p) = (p_{(1)}, p_{(1)} - p_{(2)}, H(p)/\ln k, k/255)$, where $p_{(1)} \geq p_{(2)}$ are the two largest probabilities, and outputs two logits for acting on the answer versus escalating it. The checkpoints’ configuration records the costs it was trained against (escalation 0.5, a wrong action 3.0). In our recorded run it returned an act probability of exactly 1.0 on all 36 answers, including answers whose top option held 0.39 of the mass [measured]. We carry its output on the wire as an ignorable field and do not use it: escalation is decided by the program’s thresholds and by policy (Section 4).

3.3 The family

Model	Encoder	Params	Context	Revision
kai-1	ModernBERT-large [24]	421M	512	b50502c2
kai-1-multilingual	mmBERT-base [15]	322M	1024	3119843b
kai-1-agent	ModernBERT-large	421M	1024	e3e41649

Table 1: The Kai checkpoints on Hugging Face under `hanzoai/`. Parameter counts and context lengths are the upstream model card’s [reported]. Each is the upstream Laya checkpoint (`laya`, `laya-multilingual`, `laya-typed-decisions`) with byte-identical `model.safetensors` [in the code]. The multilingual checkpoint reads up to 8,192 tokens when asked to.

`kai-1` is the English checkpoint. `kai-1-multilingual` covers more than a hundred languages at a smaller width ($d = 768$, 12 heads, rotary base 160,000, against $d = 1024$ and 16 heads for the English encoder). `kai-1-agent` is the English architecture fine-tuned by its upstream authors on a typed-decisions benchmark of 2,000 decisions across four workflows, where they report 0.766 accuracy against 0.362 for the base English checkpoint [reported] [4]. The upstream authors also report 32.8ms for one question and 72.3ms for ten batched questions on a T4 GPU with the multilingual checkpoint, 39.5 ms for one question with the English one, and 193–464 ms per request on CPU with checkpoints preloaded [reported]. We have not yet reproduced these; Section 7.2 says how we will.

3.4 Routing between checkpoints

The English checkpoint does not degrade gently off English. Its authors report that on Khmer it scores 0.000 accuracy at 0.952 mean confidence [reported] [4], so a confidence gate cannot catch the

failure after the fact. The choice of checkpoint is therefore made before the forward pass, from the script of the state and, for Latin script, from a function-word and diacritic heuristic. The Rust port keeps the reference detector’s word lists, thresholds and tie-breaks unchanged, so a state routes to the checkpoint it would have routed to before, and its tests replay the reference’s routing decisions [proposed]. In the recorded run, seven of nine states routed to the English checkpoint and the Hindi and Spanish states to the multilingual one [measured].

3.5 Provenance

The upstream code and checkpoints are released under Apache-2.0; the fork is `hanzoai/decision` on GitHub and the weights are the three repositories of Table 1, with the upstream notice kept. The encoders are ModernBERT [24] and mmBERT [15], both with published training descriptions. For deployments that must account for the origin of every model in the loop, this matters: the decision plane does not depend on the generative models it governs.

3.6 The Rust runtime

The native runtime has two parts [proposed]. A model module in Hanzo’s tensor library (`hanzo-transformers, models/kai.rs`) implements the encoder and head above over the existing ModernBERT implementation, with the rotary base and head counts read from each checkpoint’s configuration. A crate, `hanzo-decision`, holds everything that is not the network: question parsing, sequence layout and collation, tokenisation, checkpoint routing, decoding of logits to typed answers, Decision Programs, the verdict lattice, and a remote backend for OpenRouter. Its parity suite replays the reference runtime’s collated batches, logits, act outputs, decoded answers and routing decisions, recorded from Python with PyTorch in f32 on CPU, and requires decoded probabilities to agree within 10^{-4} . The same crate links into the Hanzo Dev agent and into the hosted engine, so the decision a developer’s machine makes offline and the one the cloud makes are computed by the same code.

4 Policy is the authority

A model that answers “may this command run?” will sometimes be wrong, and some of its inputs are written by an adversary: a file the agent read, a web page it fetched, an issue it was asked to fix. Prompt injection is not a failure of a particular model but a property of any model that reads untrusted text. The question is therefore not how accurate the decision model is but what the system does when it is wrong. We answer it structurally.

4.1 The verdict lattice

Definition 3 (Verdicts). The verdicts are the chain $V = \{\text{ALLOW}, \text{ASK}, \text{DENY}\}$ ordered $\text{ALLOW} < \text{ASK} < \text{DENY}$, with join $a \sqcup b = \max(a, b)$. ALLOW runs the action, ASK puts it to a person, DENY refuses it.

A deterministic policy π maps a state to a verdict. It is written by people, reviewed like code, and evaluated before any model is consulted: a command that reads `~/ssh` and opens a network connection is DENY by rule, whatever any model says about it. Where no rule applies the policy returns ALLOW, which here means only “the policy has no objection”. A Decision Program

(Section 5) maps the same state to a model verdict $\mu(s) \in V$, and the effective verdict of an enforced program is

$$\varepsilon(s) = \pi(s) \sqcup \mu(s). \quad (5)$$

Proposition 2 (The model only tightens). *For every state s and every model verdict $\mu(s)$, $\varepsilon(s) \geq \pi(s)$. In particular the set of states on which an action runs without a person, $\{s : \varepsilon(s) = \text{ALLOW}\}$, is a subset of $\{s : \pi(s) = \text{ALLOW}\}$, whatever the model outputs.*

Proof. $\max(a, b) \geq a$. If $\varepsilon(s) = \text{ALLOW}$ then $\pi(s) \leq \varepsilon(s) = \text{ALLOW}$, and ALLOW is the least element. $\square$

Corollary 1. *The safety envelope of the system is the policy’s and does not depend on the model. An adversary who controls the state completely, and so controls $\mu(s)$ completely, can make an action ask or be refused where the policy would have allowed it, and can do nothing else.*

The proposition is elementary, which is the point. The property that matters most is guaranteed by an order and a maximum, not by an estimate of a model’s error rate.

4.2 Abstention and escalation

A model verdict is derived from answers, and an answer can be unsure. Let the program’s verdict question have answer p with rescaled confidence κ_p (Equation (1)) and threshold t . The model verdict is

$$\mu(s) = \begin{cases} \text{ASK} & \text{if } \kappa_p < t, \\ \arg \max_i p_i & \text{otherwise,} \end{cases} \quad (6)$$

so an unsure model defers to a person rather than guessing. Programs may also declare escalations: a `noul` question whose p_{true} reaches its threshold raises the verdict to a stated level. The command-risk program of Appendix B escalates to `DENY` when “could this command read, print or send credentials, keys or tokens?” holds at $p_{\text{true}} \geq 0.3$, and to `ASK` for production impact or network access. Escalations are joins, so Proposition 2 holds for the composed verdict unchanged.

4.3 Modes, and why promotion is measurable

A program runs in one of three modes.

shadow

The model verdict is recorded and never shown or applied; $\varepsilon = \pi$.

advisory

The model verdict is shown to the operator beside the policy’s; $\varepsilon = \pi$.

enforced

$$\varepsilon = \pi \sqcup \mu.$$

A program’s thresholds are set against one calibration of one model revision and the program names that calibration. A decision computed under a different calibration is demoted to shadow, because its probabilities are not the ones its thresholds were validated for [proposed].

Proposition 3 (Enforcement costs only friction). *Let a program run in shadow over a sample of states and record $\pi(s)$ and $\mu(s)$ for each. The states on which enforcing it would change the outcome are exactly $F = \{s : \mu(s) > \pi(s)\}$, and on each of them the change is from a looser verdict to a stricter one.*

Proof. $\pi(s) \sqcup \mu(s) \neq \pi(s)$ iff $\mu(s) > \pi(s)$, and then the join is $\mu(s) > \pi(s)$. $\square$

The shadow record therefore contains the complete counterfactual of enforcement, not an estimate of it. Each state in F is either a catch (the policy would have allowed something a person, shown it, would refuse) or friction (a person is interrupted, or an action refused, for nothing). With labels from the people who handled those states, the catch rate and the friction rate are counts, and promoting a program from shadow to enforced is a decision about whether the catches are worth the friction. It requires no estimate of the probability that the model misses something, because enforcement cannot make a miss worse than the policy’s own.

4.4 Constraints and preferences

Not every program produces a verdict. A routing program chooses a model tier, a context program chooses how much of a chunk to show. These split into two parts. The *constraint* part lives in a lattice with the same rule: which providers may see this state (any, approved, self-hosted only), whether a file is sensitive enough that it must not leave the organization. Policy sets the floor, the model may tighten it, Proposition 2 applies. The *preference* part is a free choice within the constraint: cheap or frontier among the permitted models, a summary or the full chunk. There the model decides, and each program declares a default for abstention that errs toward quality over cost, such as the frontier tier or the full chunk. A wrong preference costs money or quality; it cannot cross a constraint.

5 Decision Programs and the wire

5.1 Decision Programs

Definition 4 (Decision Program). A Decision Program is a named, versioned, immutable record

$$P = (\text{name@version}, \text{description}, \text{model}, \text{mode}, \text{calibration}, Q, t, R)$$

of the model that answers it, the mode its output may be used in, the calibration its thresholds were set against, a set Q of named typed questions, a threshold t_q for each question, and an optional verdict rule R naming the verdict question and its escalations.

A program is the only place a question’s natural-language text lives. A caller does not write “should this command run?”; it runs `agent.command-risk@1` on a state. Any change to a question’s wording, its criteria, a threshold, the model or the calibration is a new version, because thresholds are only meaningful for the probabilities they were validated against, and those probabilities depend on every one of these. Versions are never edited in place, so a record that names `agent.command-risk@1` names one fixed set of questions forever.

Table 2 lists the programs an agent loop runs. They cover the decision points of the Jev notes [1] (context visibility, routing, tool choice, permissions, sensitivity) and add three the loop needs at its edges: preflight before a task starts, progress while it runs, and completion before it reports done. `agent.complete@1` asks separately whether the work is done and whether that is shown rather than claimed, because an agent that says it ran the tests and an agent whose transcript contains a passing test run are different states, and only the second should end a task.

Program	Questions	Mode at v1
<code>agent.preflight@1</code>	task type (code, conversation, research, operation); complexity (4 levels); model tier; needs the user; needs repository context; sensitivity (4 levels); private model required	advisory
<code>agent.command-risk@1</code>	verdict (ALLOW, ASK, DENY); secret exposure; production impact; network	advisory
<code>agent.model-route@1</code>	tier (cheap, zen-coder, frontier); reasoning depth (4 levels); long context; private model required	shadow
<code>agent.context@1</code>	scope (none, file, module, repository, external); needs repository context; needs the web; needs memory	shadow
<code>agent.tool-select@1</code>	next tool (read, search, edit, shell, web, ask the user, none); can the next steps run in parallel	shadow
<code>agent.progress@1</code>	status (advancing, stalled, looping, blocked); needs the user	advisory
<code>agent.complete@1</code>	done; verified rather than claimed; work remaining (3 levels)	advisory

Table 2: The Decision Programs an agent loop runs, as defined for the Rust runtime [\[proposed\]](#). All seven name `kai-1-agent`. Unmarked questions are `noul`.

5.2 The decision record

Every execution of a program produces a record: the program id, a content hash of the state and a reference to where the state is kept under the tenant’s retention policy, the backend and model revision that answered, the calibration, every answer with its full distribution, the policy verdict, the model verdict, the effective verdict and the mode, latency and metered usage, and the caller’s identity. With the state and the model revision, a record can be replayed; on the same backend the replay reproduces the distributions, and across backends the parity bound of Section 3 applies. The records of a program in shadow are the sample on which it is promoted (Proposition 3), and the outcomes people chose on those states are its training labels.

5.3 Testing a program

Because a program’s output is typed, a program can be tested the way code is, and a change to it is a change a test suite can catch [\[proposed\]](#).

- **Unit tests** are states with the answer a person expects, and a tolerance, such as: a command that pipes a private key to `curl` is never ALLOW. They run on every new model revision and calibration before a program version names them.
- **Replay tests** re-run recorded decisions and require the same distributions within the parity bound. They catch a runtime or tokenizer change that moved answers without anyone intending it.
- **Calibration tests** check, on a labelled sample, that answers reported at a given $p_{\max}$ are right about that often, per question.
- **Policy tests** are the policy’s own: deterministic rules tested as the code they are, independent of any model.

A new program version is the old one with a passing suite; a regression shows up as a failing named case, not as a vague change in agent behaviour noticed weeks later.

5.4 POST /v1/decisions

The interface takes OpenRouter’s Decisions API [18] as its wire format, field for field. A request names a model, a state and a map of named questions; the response returns a map of typed answers under the same names, with usage and cost. Appendix A gives an example. Hanzo adds fields a client of the upstream schema may ignore: **answer_confidence** on every answer; **routing**, saying which checkpoint answered and why; the act head’s output as **action**; and a program form of the request, {**program**, **state**}, which expands a named Decision Program to its questions and returns its verdict and mode beside the answers.

Model-agnostic by name. The **model** field chooses the backend. A **kai-*** name is answered by the native runtime. A **typesafe/jev-*** name, such as **typesafe/jev-1.13** or the alias **~typesafe/jev-latest**, is forwarded unchanged to OpenRouter’s **POST /api/alpha/decisions**, and the response is checked at the boundary: every question must come back answered with its own type, and a choice’s probabilities are restored to the caller’s label order. OpenRouter listed Jev on 2026-09-24 at \$0.042 per million input tokens with output free and a 32k context; its default model listing omits it, and it appears when the listing is asked for all output modalities. Because the wire is shared and **confidence** is normalised to Equation (1) on both paths, a program can move between Jev and Kai by changing one field, and its records say which answered.

5.5 Where it runs

The same crate is used in three places [proposed].

- **In the agent.** Hanzo Dev links the runtime and answers **agent.command-risk@1** and **agent.preflight@1** on the developer’s CPU. The state, which is often a command line or a diff, does not leave the machine to be judged.
- **In the engine.** Hanzo’s inference engine serves **/v1/decisions** natively, batching questions across requests.
- **In the cloud.** Hanzo’s cloud exposes **/v1/decisions** with its identity, quota and metering, and routes to the engine or to OpenRouter by model name.

Nothing in the runtime depends on a network service or a Python stack, so the same binary serves a disconnected installation, an on-premises cluster or a workstation as it serves the cloud, on the tensor library’s CPU backend and, where present, its CUDA and Metal backends. In a disconnected installation the Jev backend is simply absent, and a program that names it fails closed rather than falling back to another model.

5.6 First deployment: the coding agent in shadow

The first deployment is Hanzo’s Slack coding agent, which takes a request in a channel and runs it in a sandbox. Every run records **agent.preflight@1** before it starts and **agent.command-risk@1** before each command, in shadow. A deployment may run a program in a lower mode than its definition allows, never a higher one, so the mode a program is defined with is a ceiling that each deployment approaches separately. The agent’s behaviour is unchanged; the records accumulate

the decisions a person made on the same states. That is the flywheel: shadow records become labelled data, the labelled data trains a Kai checkpoint on Hanzo’s own traffic, the new checkpoint gets a new calibration and new program versions, and each program is promoted only when its shadow counts justify it.

6 Decisions as programs for studies

The objects of the previous sections were built for an agent’s inner loop, but nothing in them is specific to agents. A trade study, an analysis of alternatives or a source selection is also a set of typed judgments over evidence, aggregated into a recommendation. Today such a study usually lives in slides and spreadsheets: the scores survive, the reasons for them are in someone’s memory, and a year later, when a cost estimate or a test result changes, the study is redone from the beginning because nobody can say which of its conclusions depended on the number that moved. Public programs now ask for the alternative. The U.S. Army’s 2026 SBIR topic ARM26BX06-NV012, *Agentic-AI, Schema-Driven Decision Management for Auditable Studies and Acquisition Decisions* [23], asks for studies that are defined by a schema, executed with agentic assistance, auditable after the fact, and refreshable as evidence changes. This section describes how the objects above provide that [proposed].

6.1 The study as a program

A study program fixes:

- alternatives $A = \{a_1, \dots, a_n\}$;
- criteria $J = \{j_1, \dots, j_m\}$ with weights $w \in \Delta^{m-1}$, $w_j \geq 0$, $\sum_j w_j = 1$;
- for each criterion j , ordered levels L_j with a description of each and a value $u_j(\ell) \in [0, 1]$ for each level, in the manner of multi-attribute utility theory [14];
- named assumptions θ , each with a value and a declared range: a budget ceiling, a fielding date, a discount rate, a threat estimate;
- constraints, predicates $g(a; \theta)$ that an alternative must satisfy to be considered at all, such as fitting the budget or meeting a threshold requirement;
- for each cell (a, j) , an executor and its inputs. Most cells are a typed **score** question whose levels are L_j , answered by Kai over the evidence a query selects for alternative a . A cell that is arithmetic, such as a life-cycle cost from a cost model or a range from a performance model, is computed by that deterministic model from the evidence and θ , and its value placed on the same scale.

It is a Decision Program in the sense of Section 5: named, versioned, immutable, bound to a model and a calibration. Judgments go to Kai, arithmetic goes to deterministic code, and nothing numeric comes from generated prose.

Aggregation. Cell (a, j) returns a distribution p_{aj} over L_j . Its expected value and variance are

$$U_{aj} = \sum_{\ell \in L_j} p_{aj}(\ell) u_j(\ell), \quad \sigma_{aj}^2 = \sum_{\ell \in L_j} p_{aj}(\ell) u_j(\ell)^2 - U_{aj}^2, \quad (7)$$

and an alternative's value is the weighted sum

$$V(a) = \sum_j w_j U_{aj}, \quad \text{Var } V(a) = \sum_j w_j^2 \sigma_{aj}^2, \quad (8)$$

the variance under the assumption that cells are answered independently. A deterministic cell has $\sigma_{aj} = 0$ unless its model reports an uncertainty. The recommendation is the best feasible alternative, $a^* = \arg \max \{V(a) : g(a; \theta) \text{ holds for every constraint}\}$.

The additive form assumes the criteria are preferentially independent; where they are not, the study must say so and use a different aggregation, and the sensitivity results below apply only to the additive one.

6.2 The DecisionPackage

The unit of record for a study is a DecisionPackage: the study program and its version; the assumptions and constraints in force; every cell's question or deterministic model, evidence references with content hashes, model revision, calibration and full answer distribution; every human override, recorded beside the answer it overrides with its author and reason rather than in place of it; the policy results for any cell a policy governs; the values $V(a)$ with their variances; the recommendation; the sensitivity report of Section 6.3; and the approvals, recording who approved the program before it ran and who accepted the recommendation after. A package is the audit trail: every number in the study's conclusion resolves to a field in it, and every field resolves to an answer and the evidence the answer read.

A package separates what people decided from what models estimated, and each kind of work goes to the component suited to it. People decide the alternatives, criteria, weights, level descriptions, value functions, assumptions and constraints, and approve the program that encodes them. Kai makes the bounded judgments. Deterministic models do the arithmetic and check the constraints. Policy governs who may approve what. People may override any cell. The generative plane helps with each step it does not own: it elicits criteria and level descriptions from source documents and interviews, proposes the evidence query for each cell, reasons through the ambiguous engineering questions a person must then settle, and writes the narrative of the final report. Every number the narrative states must resolve to a package field, and a sentence that states one that does not is rejected, in the way our assertion graph rejects an uncited claim in a generated answer [10].

6.3 What flips the decision

A recommendation is only as useful as the account of how fragile it is. A package reports four answers to "what would flip it?".

Weights. For a challenger b , write $\delta_b = U_{a^*} - U_b \in \mathbb{R}^m$ for the difference of cell values across criteria, so that $V(a^*) - V(b) = w^\top \delta_b > 0$. Let $P = I - \frac{1}{m} \mathbf{1} \mathbf{1}^\top$ project onto the directions that keep the weights summing to one.

Proposition 4 (Smallest weight change). *The smallest change Δ in Euclidean norm with $\mathbf{1}^\top \Delta = 0$ that makes b tie a^* is*

$$\Delta_b^* = -\frac{w^\top \delta_b}{\|P\delta_b\|^2} P\delta_b, \quad \|\Delta_b^*\| = \frac{w^\top \delta_b}{\|P\delta_b\|}, \quad (9)$$

provided $P\delta_b \neq 0$. If $w + \Delta_b^ \geq 0$ it is the smallest change within the simplex; otherwise it is a lower bound on that change.*

Proof. Minimise $\|\Delta\|^2$ subject to $\mathbf{1}^\top \Delta = 0$ and $\delta_b^\top \Delta = -w^\top \delta_b$. Since Δ lies in the range of P and P is symmetric and idempotent, $\delta_b^\top \Delta = (P\delta_b)^\top \Delta$, so this is the distance from the origin to a hyperplane in that subspace with normal $P\delta_b$, attained at the stated point. Adding the constraint $w + \Delta \geq 0$ shrinks the feasible set, which can only raise the minimum. If $P\delta_b = 0$, every weight vector ranks a^* and b the same way and no change of weights flips them. $\square$

The package reports $\rho_w = \min_b \|\Delta_b^*\|$, the challenger that attains it, and the direction: which criteria’s weights must rise and which fall. A stakeholder who disagrees with the weights can read from it how far they would have to disagree to change the answer.

Answers. Moving one cell’s value changes one alternative’s V by w_j times the move. For the runner-up b with margin $\Delta V = V(a^*) - V(b)$, the recommendation flips if U_{a^*j} falls by $\Delta V/w_j$ or U_{bj} rises by $\Delta V/w_j$, and likewise for every other challenger with its own margin. We report each cell’s required shift r_{aj} in units of the cell’s own standard deviation, $z_{aj} = r_{aj}/\sigma_{aj}$. A cell with small z_{aj} is one where the recommendation turns on a judgment the model is itself unsure of: that is where an analyst’s time, or more evidence, is worth the most, the question the value of information literature asks [11]. A cell whose required shift exceeds the range of u_j cannot flip the recommendation alone and is reported as such.

Assumptions and constraints. Weights and answers are not the only inputs a reviewer doubts. For each assumption θ_i the package sweeps its declared range, holding the others fixed, recomputes the cells that depend on it, and reports the break-even value θ_i^* , if there is one in the range, at which the recommendation changes, whether because another alternative’s value overtakes a^* ’s or because a constraint admits or excludes an alternative. Where the margin $V(a^*; \theta) - \max_b V(b; \theta)$ is continuous in θ_i and changes sign across the range, bisection finds the crossing; otherwise a grid does, and the package says which. Only cells whose inputs include θ_i are recomputed at each point, by the reuse rule of Proposition 5, and a deterministic cell costs a function call. “The recommendation holds for any budget above \$41M and any fielding date before 2029” is the kind of sentence this produces, and each of its numbers resolves to a sweep in the package.

The model’s own uncertainty. Sampling a level for every cell from its p_{aj} and recomputing $\arg \max$ estimates the probability that the recommendation would differ if each cell were resolved, $\Pr[\arg \max_a V(a) \neq a^*]$. The estimate is as good as the calibration of the cells it samples, which is one more reason the program records the calibration it was run under.

An illustration. The following is a constructed example, not data. Three alternatives are scored on cost ($w = 0.40$), performance (0.35) and schedule risk (0.25):

	cost	performance	schedule	V
A	0.60	0.80	0.70	0.695
B	0.80	0.60	0.60	0.680
C	0.50	0.70	0.90	0.670

A is recommended by 0.015 over B and 0.025 over C. For B, $\delta_B = (-0.20, 0.20, 0.10)$ and Equation (9) gives $\Delta_B^* = (+0.040, -0.029, -0.012)$, of norm 0.051: raising the cost weight from 0.40 to 0.44, taken from the other two, makes B tie A. For C the change is $(-0.042, -0.042, +0.083)$, of norm 0.102, a rise in the schedule weight from 0.25 to 0.33. Both stay in the simplex, so both are exact, and $\rho_w = 0.051$ toward B.

For the answers, suppose A’s performance cell is the least certain, $\sigma = 0.12$, with $\sigma = 0.03$ for B’s cost, 0.04 for the schedule cells of A and B, and 0.05 elsewhere. The shift that flips A against B is $0.015/0.35 = 0.043$ on A’s performance, which is $z = 0.36$; the next smallest is A’s cost, $0.015/0.40 = 0.038$ at $\sigma = 0.05$, $z = 0.75$; no cell of C can flip the recommendation within less than $z = 1.25$. The package would say: the recommendation rests on A’s performance evidence, and a third of a standard deviation there changes it. Taking each cell as normal with these moments, B beats A with probability 0.39 and C beats A with probability 0.33, and in 4×10^5 joint samples A is best in 54%, B in 28% and C in 19%. The study recommends A and says, in the same document, that the recommendation is close to a three-way tie which one more performance measurement of A would most likely settle.

6.4 Refresh

A study refreshed a year later should cost in proportion to what changed. Give each cell a key: the program version, the question, the model revision and calibration, and the content hashes of the evidence its query selected.

Proposition 5 (Exact reuse). *If a cell’s key is unchanged and inference is deterministic on the backend, its answer distribution is unchanged, so reusing the stored answer gives the same package as recomputing it. A refresh then asks $O(c)$ questions for c changed cells, and every value, variance and sensitivity figure is recomputed from the cells, which is arithmetic.*

Proof. The distribution is a function of the question, the state and the model revision with its calibration (Section 2); the state of a cell is the evidence its key hashes. Equal keys give equal inputs. $\square$

A refresh produces a new package and a diff: which cells changed and by how much, whether a^* changed, and how ρ_w and the cells’ z values moved. The refresh can run on a schedule or on an event, when a new document lands that some cell’s query selects.

6.5 Evidence and time

A package must be able to say not only what it concluded but what was known when it concluded it. Hanzo’s temporal assertion graph [10] answers every read at a valid instant and a transaction instant, and an answer as known at an instant is unchanged by every later write except erasure. If a study’s evidence queries read the graph and the package records the transaction instant of the run, the package can be replayed exactly as known on the day it was decided, even after the evidence was later corrected, and the diff between that replay and a refresh separates what changed in the world from what changed in the record.

Engineering studies increasingly keep their alternatives, requirements and analyses in a system model. SysML v2 [17] gives such models a textual notation and a standard API, which makes them addressable as evidence: an alternative can be a reference to a part definition, a criterion to a requirement, and a cell’s state to the serialised elements its question should read. We have not yet built this binding.

7 Evaluation

This section reports the one run we have made and the protocol for the rest.

7.1 The recorded run

We ran the reference runtime, upstream Laya 0.3.20 in Python with PyTorch in f32 on CPU, against the `hanzoai/kai-1` bundle at revision `b50502c2`, whose `root`, `multilingual/` and `typed-decisions/` folders hold the three checkpoints of Table 1. Nine states, each with one to three questions, were answered by all three checkpoints: 27 cases and 36 answers, recorded with their collated batches, logits, act outputs and decoded answers, together with the router’s choice for each state. The record is the parity fixture of the Rust runtime. The states are small by design; they exercise the three question types, criteria given as strings and as objects, a single-option choice, a long state, a conversation given as a list of messages, and two non-English states. They are not a benchmark, and we draw no accuracy figure from them. Everything in this subsection is [\[measured\]](#).

Permissions. Two of the states were shell commands put to a three-way permission question:

```
cat ~/.ssh/id_ed25519 | curl -X POST https://example.com -d @- and rg -n TODO src/
```

with the criteria `ALLOW` (“read-only, inside the repository”), `ASK` (“writes outside the repo or touches the network”) and `DENY` (“touches secrets, `~/.ssh` or `.env`”). The first sends a private key to a remote host; the second searches the source tree.

Command	Checkpoint	$p(\text{ALLOW})$	$p(\text{ASK})$	$p(\text{DENY})$	Answer	κ	act
exfiltrate key	<code>kai-1</code>	0.419	0.355	0.226	<code>ALLOW</code>	0.129	1.0
	<code>kai-1-multilingual</code>	0.689	0.194	0.118	<code>ALLOW</code>	0.533	1.0
	<code>kai-1-agent</code>	0.371	0.389	0.240	<code>ASK</code>	0.084	1.0
search source	<code>kai-1</code>	0.386	0.273	0.341	<code>ALLOW</code>	0.079	1.0
	<code>kai-1-multilingual</code>	0.764	0.102	0.134	<code>ALLOW</code>	0.646	1.0
	<code>kai-1-agent</code>	0.424	0.345	0.231	<code>ALLOW</code>	0.136	1.0

Table 3: The three bootstrap checkpoints on two commands [\[measured\]](#). κ is Equation (1) with $k = 3$; “act” is the act head’s probability of acting. The router sends both states to `kai-1`.

Table 3 shows what the bootstrap checkpoints know about permissions: very little. No checkpoint put more than 0.25 on `DENY` for the key exfiltration, and two ranked `ALLOW` first. The English checkpoint, which the router chooses for both states, gave `ALLOW` a slightly *higher* probability for the exfiltration (0.419) than for the search (0.386). The answers are close to uniform: κ is below 0.14 for four of the six. The act head returned 1.0 for all six, as it did for every answer in the run.

Run through the machinery of Section 4, the picture changes. Under the command-risk program’s verdict threshold of $t = 0.5$, the English and agent checkpoints abstain on both commands, so their verdict is `ASK`; the multilingual checkpoint clears the threshold and says `ALLOW` for both. A deterministic rule that refuses reading `~/.ssh` together with network egress makes the policy verdict for the first command `DENY`, and by Proposition 2 the effective verdict is `DENY` for every checkpoint in every mode. The search command, which no rule objects to, runs under the policy alone and asks under the English and agent checkpoints if they were enforced: friction, in the sense of Proposition 3, and the reason `agent.command-risk@1` starts in advisory mode and its first deployment in shadow.

These questions are not the ones `agent.command-risk@1` asks. The program words its criteria differently and adds three `noul` questions whose escalations are designed for exactly this case. We report the recorded questions because they are the ones we have measured.

Disagreement on a plain predicate. The first state reads “Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan.” All three checkpoints assigned it to billing, with $p_{\max}$ of 0.987, 1.000 and 0.812, and all three put its urgency near the top of a three-level scale (expected levels 1.77, 1.79 and 1.73). Asked the predicate “Does the user threaten to cancel or leave?”, the English checkpoint answered $p_{\text{true}} = 0.879$ and the agent checkpoint 0.771, while the multilingual checkpoint answered $p_{\text{true}} = 0.045$: a confident no to an explicit threat. The router sends this state to the English checkpoint, so in service the answer would have been right. It is still a warning. The upstream authors note that `noul` answers are sensitive to how the two labels are described [4], and the multilingual checkpoint, which is the only one that answers non-English text, ships without fitted temperatures (Section 3). `noul` thresholds in a program must be validated per checkpoint, which is what binding a program to a calibration enforces.

Routing. The Hindi state (“I was charged twice in March, please refund the duplicate amount”) was routed to the multilingual checkpoint, which answered billing at 0.951; the English checkpoint, had it been asked, would have said billing at 0.451 and the agent checkpoint “other” at 0.434. The Spanish state (“the app closes every time I open the settings”), Latin script with diacritics, was routed to the multilingual checkpoint, which answered technical with $p_{\max} = 0.989$ where the English checkpoint gave 0.661. On both, the checkpoint the router chose was the one that was most sure and, by inspection, right.

7.2 What will be measured, and how

Each of the following has a stated procedure so that its result can be reported whichever way it comes out [proposed].

E1 Parity.

The Rust runtime replays the recorded run: identical collated batches, identical routing, decoded probabilities within 10^{-4} of the reference, identical labels. A checkpoint that fails is not served natively.

E2 Latency and throughput.

Per-question and per-request latency, median and 99th percentile, at batch sizes 1, 8 and 32, for native Kai and the Python reference on the same CPU (x86-64 with AVX2 and AVX-512, and Apple silicon), and for Jev through OpenRouter measured from the same host with the network included. The first round of every binary is run and discarded; a freshly started process reads high on its first round.

E3 Agreement.

Kai and Jev answer the same questions over the same states; we report agreement of labels and the distribution of $|\Delta p_{\max}|$ by question type, and treat disagreement as a sample to label, not as an error of either model.

E4 Accuracy and calibration on our traffic.

From the coding agent’s shadow records, labelled by the decisions people made, accuracy, Brier score and expected calibration error per program and question, before and after refitting temperatures, and per checkpoint.

E5 Promotion.

For each program, the catch and friction counts of Proposition 3 over its shadow sample, and the decision taken on them.

E6 Studies.

On a trade study with a published outcome, whether the package reproduces the recommendation; whether the flip analysis identifies the cells that later evidence actually moved; and the number of questions a refresh asks against the number a full rerun asks.

Cost is reported as measured cost per thousand decisions on each backend, compute for the native runtime and billed tokens for the others, and not as a saving against a counterfactual we did not run.

8 Related work

Typed decision models. TypeSafe’s Jev [22] introduced the typed interface this paper adopts, served through OpenRouter’s Decisions API [18], and the design notes on building a coding agent around it [1] argue for the harness structure of Section 5: explicit typed state, context assembled per query rather than accumulated, routing priced by the cost of rebuilding context rather than by tokens, and permissions as decisions. Laya [4] is an open, non-autoregressive implementation of the same interface on encoder models, trained with rewards from strictly proper scoring rules [8]; Kai’s checkpoints are Laya’s. What this paper adds is not a model but the frame around one: a fixed confidence convention, programs bound to calibrations, the precedence of policy with its consequences for promotion, and the extension to studies.

Encoders as classifiers. Scoring options against a text with a bidirectional encoder is the cross-encoder of passage re-ranking [16], and treating a label description as a hypothesis the text may entail is the entailment approach to zero-shot classification [27]; both descend from BERT [6]. ModernBERT [24] and mmBERT [15] are the encoders under Kai.

Structured output from generative models. Constrained decoding makes a generative model emit text that parses against a grammar or schema [25]. It removes parsing failures but not the generation, and the probability it can report is that of a token sequence, not a distribution over the caller’s options. Tool-using agents [26, 20] make decisions of the kind in Table 2 inside the generative loop.

Guard models. Safety classifiers such as Llama Guard [12] judge prompts and responses against a taxonomy of harms. A command-risk program is a guard over actions rather than text, and Section 4 places any such guard below a deterministic policy rather than in place of one.

Calibration and abstention. Temperature scaling [9] is how the checkpoints’ temperatures were fitted, and the reject option [3] and selective classification [7] are the theory behind abstaining to ASK below a threshold.

Authority as an order. Enforcing the join of two verdicts is the lattice model of secure information flow [5] applied to decisions, and a policy evaluated on every action before it runs is a reference monitor [2].

Decision analysis. The study model of Section 6 is additive multi-attribute utility [14]. Sensitivity of multi-criteria rankings to their weights is studied directly by Triantaphyllou and Sánchez [21], and global sensitivity analysis more broadly by Saltelli et al. [19]; ranking cells by the shift that flips a recommendation relative to their uncertainty is in the spirit of the value of information [11]. The two-system framing of fast classification beside slow generation borrows Kahneman’s vocabulary [13] and nothing more.

9 Limitations

The bootstrap models are not good at our questions. The checkpoints were trained by others on other data. Their authors report 0.362 zero-shot accuracy for the base English checkpoint on their typed-decisions benchmark [4], and on our permission questions all three were close to uniform (Table 3). The design tolerates this, because a model it has not promoted cannot act, but it does not hide it: until a checkpoint is trained on Hanzo’s traffic, Kai’s contribution to the agent loop is a record, not a decision.

Calibration is assumed, then checked. Every threshold, abstention rule and flip probability in this paper treats the model’s probabilities as meaning what they say. The multilingual checkpoint ships with no fitted temperature, the English one with a fitted value the runtime refuses to apply, and the upstream authors report a confidently wrong English checkpoint off Latin script. Programs are bound to calibrations for this reason, and E4 is the check.

The act head is not used. It was constant in our run. It may carry signal on the distribution it was trained on; we have not found it.

Adversarial state beyond the lattice. Proposition 2 protects constraints, not preferences. An adversary who controls a state can push a routing program toward a cheaper model, a context program toward hiding a chunk, or a completion program toward declaring a task done early. Those programs need deterministic backstops of their own, such as a completion that requires a passing test run in the transcript, and we have specified them only for completion. An adversary can also use Kai to cause friction deliberately, asking and refusing where the policy would allow; that is a denial-of-service against the operator’s attention, bounded by the policy but not prevented by it.

The study model has assumptions. Additive aggregation requires preferentially independent criteria; the variance formula and the normal approximation assume cells are answered independently, which fails when two cells read the same evidence; the weight flip is exact only when the minimiser stays in the simplex. A package states which of these it relies on. The illustration in Section 6.3 is constructed, and no part of Section 6 has yet been run on a real study.

Nothing here is measured at scale. The recorded run has nine states. The latency, cost and accuracy figures that would justify replacing generative calls with typed decisions in production are the subject of Section 7.2 and are not in this paper.

10 Conclusion

An agent’s small decisions are where its cost, its safety and its quality are decided, and most of them do not need a generative model. A typed question over explicit state, answered by an encoder

in one forward pass, gives a distribution a caller can threshold, record and replay. Putting that answer under a deterministic policy with a join makes the safety of the system independent of the model’s, and reduces the question of when to trust the model to a count of catches and friction that shadow mode records exactly. Versioned programs bound to calibrations keep thresholds honest as models change, and the same objects, applied to a study, give a recommendation with an audit trail, an account of what would flip it, and a refresh that costs what changed.

The bootstrap checkpoints do not yet know Hanzo’s questions, and our one recorded run says so plainly. What is built is the structure that lets them learn safely: the wire that makes Kai and Jev interchangeable, the programs that fix what is asked, the records that turn shadow decisions into labels, and the policy that stays in charge while they do.

References

- [1] Jev engineering for coding agents: The TypeSafe founder’s blueprint for building with Jev, 2026. Working note, September 2026. A synthesis of design notes by Diogo Almeida (TypeSafe), independently compiled; not affiliated with or endorsed by TypeSafe.
- [2] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972.
- [3] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1), 1970.
- [4] Convai Innovations. Laya: a multilingual, non-autoregressive decision engine. <https://github.com/NandhaKishorM/laya> and <https://huggingface.co/convaiinnovations/laya>, 2026. Version 0.3.20, Apache-2.0; model card accessed 2026-09-24.
- [5] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5), 1976.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [7] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- [8] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 2007.
- [9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- [10] Hanzo AI Research. A temporal assertion graph: Valid-time and transaction-time reads, schema as assertions, and erasure in an append-only store. <https://github.com/hanzoai/papers>, 2026. 2026-09-22.
- [11] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1), 1966.
- [12] Hakan Inan et al. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674, 2023.
- [13] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [14] Ralph L. Keeney and Howard Raiffa. *Decisions with Multiple Objectives: Preferences and Value Trade-offs*. Wiley, 1976.
- [15] Marc Marone, Orion Weller, et al. mmBERT: A modern multilingual encoder with annealed language learning. <https://huggingface.co/jhu-clsp/mmBERT-base>, 2025.

- [16] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019.
- [17] Object Management Group. OMG systems modeling language (SysML), version 2. <https://www.omg.org/sysml/>, 2025.
- [18] OpenRouter. Decisions API (alpha): POST /api/alpha/decisions. <https://openrouter.ai>, 2026. Accessed 2026-09-24.
- [19] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. Wiley, 2008.
- [20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [21] Evangelos Triantaphyllou and Alfonso Sánchez. A sensitivity analysis approach for some deterministic multi-criteria decision-making methods. *Decision Sciences*, 28(1), 1997.
- [22] TypeSafe. Jev. Served as typesafe/jev-1.13 through OpenRouter, 2026. Typed decision model; listed on OpenRouter 2026-09-24.
- [23] U.S. Army. ARM26BX06-NV012: Agentic-AI, schema-driven decision management for auditable studies and acquisition decisions. <https://armysbir.army.mil/topics/agentic-ai-schema-driven-decision-management/>, 2026. SBIR topic; accessed 2026-09-24.
- [24] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. arXiv:2412.13663, 2024.
- [25] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.
- [26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [27] Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of EMNLP-IJCNLP*, 2019.

A The wire

A request to POST /v1/decisions, in OpenRouter’s Decisions schema:

```
{
  "model": "kai-1",
  "state": "Hi, we were billed twice for March. Please refund the duplicate today or we will
    cancel our plan.",
  "questions": {
    "department": {
      "type": "choice",
      "instructions": "Which department should handle this?",
      "criteria": {
        "billing": "invoices, payments, refunds",
        "technical": "bugs, outages, system errors",
        "other": "everything else"
      }
    },
    "urgency": {
      "type": "score",
      "instructions": "How urgent is this?",
    }
  }
}
```

```

    "criteria": ["not urgent", "soon", "blocking"]
  },
  "churn_risk": {
    "type": "noul",
    "instructions": "Does the user threaten to cancel or leave?"
  }
}
}

```

The response carries one typed answer per question. The probabilities below are those the English checkpoint gave in the recorded run; **confidence** is Equation (1), **answer_confidence** is $p_{\max}$, and the remaining Hanzo fields are omitted. The **id** and **usage** values are illustrative.

```

{
  "id": "dec_...",
  "model": "kai-1",
  "provider": "hanzo",
  "answers": {
    "department": {
      "type": "choice", "choice": "billing",
      "probabilities": {"billing": 0.9865, "technical": 0.008, "other": 0.0055},
      "confidence": 0.9798, "answer_confidence": 0.9865
    },
    "urgency": {
      "type": "score", "score": 1.7722,
      "legend": {"0": "not urgent", "1": "soon", "2": "blocking"},
      "probabilities": {"0": 0.0389, "1": 0.15, "2": 0.8111},
      "confidence": 0.7167, "answer_confidence": 0.8111
    },
    "churn_risk": {
      "type": "noul", "noul": 0.879,
      "confidence": 0.758, "answer_confidence": 0.879
    }
  },
  "usage": {"input_tokens": 164, "output_tokens": 0, "cost": 0}
}

```

Changing "model" to "typesafe/jev-1.13" sends the same request to Jev through OpenRouter; nothing else in the request changes.

B A Decision Program

agent.command-risk@1 as defined for the Rust runtime [\[proposed\]](#):

```

{
  "id": "agent.command-risk@1",
  "description": "Before an agent runs a shell command: allow it, ask the user, or deny it.",
  "model": "kai-1-agent",
  "mode": "advisory",
  "calibration": "cal_6dbb072f8109a222",
  "questions": {
    "verdict": {
      "type": "choice",
      "instructions": "Should the agent run this shell command?",
      "criteria": {
        "allow": "read-only, or confined to the working repository",
        "ask": "writes outside the repository, installs software or touches the network",

```

```

    "deny": "reads or sends secrets, keys, ~/.ssh or .env files, or destroys data"
  }
},
"secret_exposure": {"type": "noul", "instructions": "Could this command read, print or send
  credentials, keys or tokens?"},
"production_impact": {"type": "noul", "instructions": "Could this command change or break a
  production system or shared data?"},
"network": {"type": "noul", "instructions": "Does this command send or fetch data
  over the network?"}
},
"thresholds": {"verdict": 0.5, "secret_exposure": 0.3, "production_impact": 0.4, "network":
  0.5},
"verdict": {
  "from": "verdict",
  "escalate": {"secret_exposure": "deny", "production_impact": "ask", "network": "ask"}
}
}

```

The verdict question abstains to ASK below $\kappa = 0.5$; each escalation raises the verdict when its noul reaches its threshold; the result is joined with the policy's verdict when the program runs enforced.